

SMARTSAFE

Informe Técnico del Proyecto

*Módulo backend, base de datos, control de versiones
y evaluación frente a la rúbrica de curso*

Registro de eventos de calle para seguridad ciudadana
Curso de doctorado — Proyecto grupal (4 integrantes)
Aplicación desplegada en: daii.unsaac.edu.pe/smartsafe

Módulo: Backend Node.js + Express + MariaDB · Rama: feature/rony-modulos

Índice

1. Introducción.....	3
1.1 Condiciones de entrega.....	3
2. Arquitectura y stack tecnológico.....	3
2.1 Tecnologías utilizadas	3
2.2 Diagrama de flujo de una petición.....	3
3. Estructura del repositorio (árbol de carpetas)	4
3.1 Árbol raíz del repositorio.....	4
3.2 Árbol del módulo backend/	4
4. Git — árbol de ramas y flujo de trabajo	5
4.1 Árbol de ramas del repositorio	5
4.2 Comandos de Git utilizados en este módulo	5
4.3 Comandos para que el equipo clone y configure el proyecto.....	6
5. Base de datos.....	6
5.1 Tablas utilizadas por el módulo backend	6
Tabla tUsuario.....	6
Tabla tEvento	7
5.2 Capa de conexión (pool reutilizable)	7
6. Código fuente del módulo backend	8
6.1 Capa de servicio — services/evento.service.js	8
6.2 Capa de rutas — routes/index.js.....	9
7. API REST — documentación de endpoints	9
7.1 Ejemplo de respuesta exitosa de /api/eventos.....	10
8. Pruebas unitarias	10
8.1 Comando para ejecutar las pruebas.....	10
8.2 Casos cubiertos (9 en total)	10
8.3 Resultado de la última ejecución	10
9. Integración continua (GitHub Actions)	11
10. Despliegue	12
10.1 Estado actual del despliegue	12
11. Rúbrica de evaluación — estado actual.....	12
11.1 Nombres Reveladores (~85%).....	12
11.2 Patrones de Diseño (~60%)	12
11.3 Pruebas Automatizadas (~55-60%)	12
11.4 Control de Versiones y Entrega (~55-60%)	12
11.5 Planificación y Equipo (sin datos suficientes)	13
11.6 Optimización (no aplica aún)	13
11.7 Integración API (~75-80%)	13
12. Pendientes y recomendaciones para el equipo	13
13. Conclusiones	13

1. Introducción

Este informe documenta el módulo backend de SmartSafe, desarrollado como parte de un proyecto grupal del curso de doctorado. El objetivo es que los cuatro integrantes del equipo compartan un mismo lenguaje técnico sobre la arquitectura, el flujo de trabajo en Git, la base de datos, las pruebas automatizadas y el estado actual del proyecto frente a la rúbrica de evaluación.

El encargo del docente fue deliberadamente acotado: un aplicativo web simple de registro de eventos de calle, donde el usuario toma una fotografía, esta se asocia a sus coordenadas GPS, ingresa su DNI y describe el evento. El reporte se envía a una base de datos de seguridad ciudadana. No se contemplan módulos de mantenimiento, reportes ni consultas.

1.1 Condiciones de entrega

- Trabajo grupal: 4 integrantes.
- Repositorio único en GitHub, con una rama de trabajo por integrante.
- La rúbrica del curso exige pruebas unitarias automatizadas.
- La aplicación debe estar (y ya está) desplegada en un servidor accesible: daii.unsaac.edu.pe/smartsafe.

2. Arquitectura y stack tecnológico

El backend sigue una estructura tipo Express Generator, con separación progresiva en capas (rutas → servicio → acceso a datos).

2.1 Tecnologías utilizadas

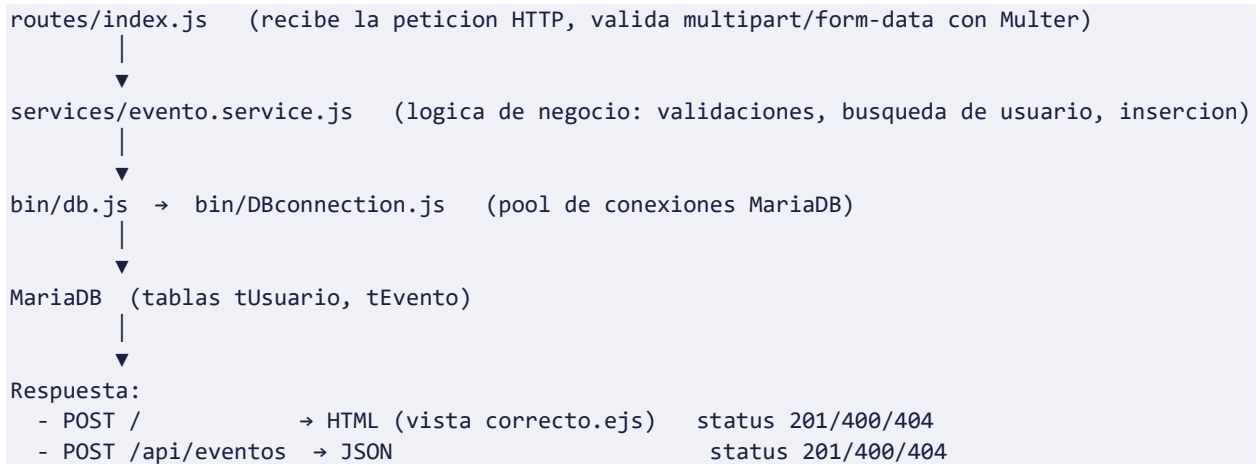
Capa	Tecnología	Uso en el proyecto
Servidor web	Node.js + Express 4	Enrutamiento HTTP, middlewares, vistas
Motor de vistas	EJS	Formulario de reporte y páginas de confirmación/error
Base de datos	MariaDB (driver mariadb 3.3.1)	Persistencia de usuarios y eventos, tipo geoespacial POINT
Carga de archivos	Multer	Recepción y almacenamiento de la fotografía del evento
Variables de entorno	dotenv	Credenciales de BD, puerto, configuración
Pruebas	Jest + Supertest	Pruebas unitarias sobre los endpoints, con BD simulada (mock)
Integración continua	GitHub Actions	Ejecuta las pruebas automáticamente en cada push/PR
Gestor de procesos	PM2	Mantiene el proceso Node.js activo en el servidor de despliegue
Control de versiones	Git + GitHub	Ramas por integrante, Pull Requests

2.2 Diagrama de flujo de una petición

Flujo: registrar evento (web o API)

Cliente (navegador o app externa)

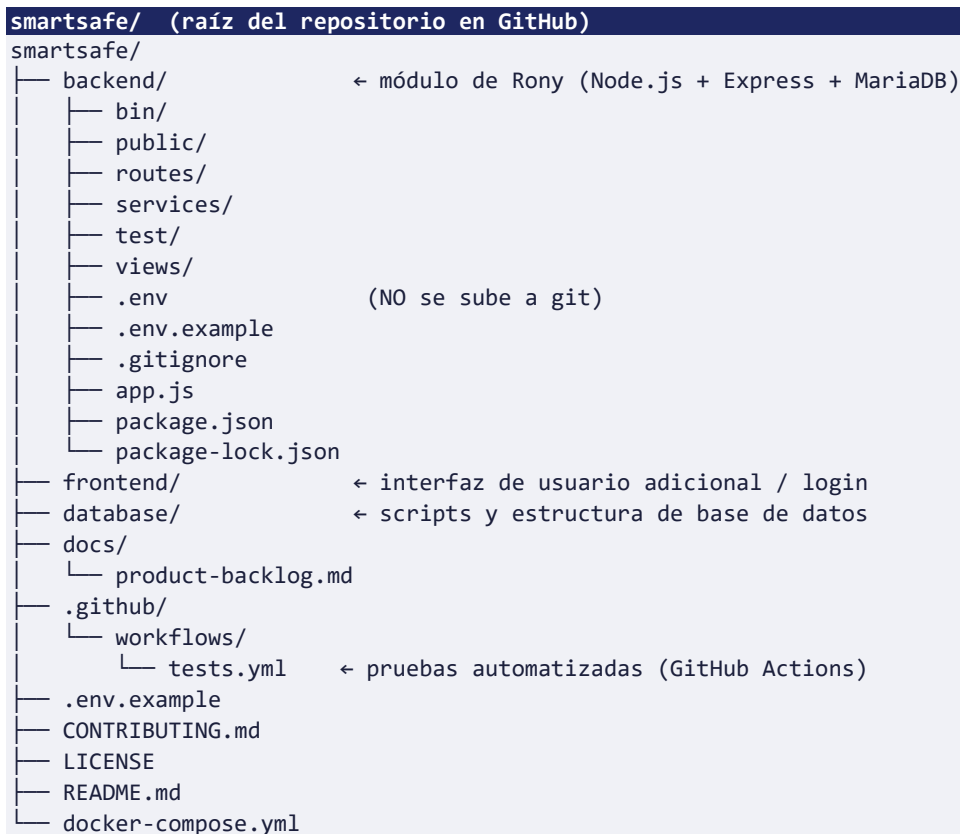




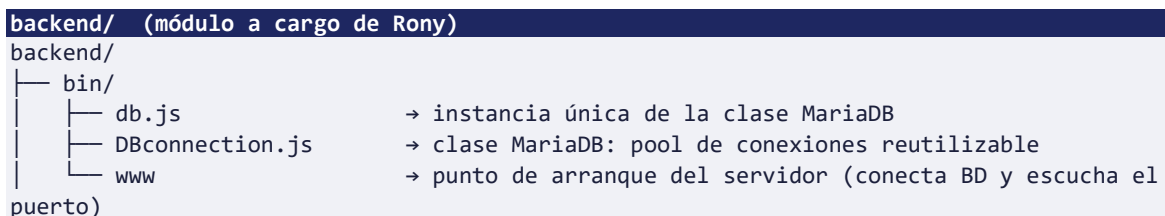
3. Estructura del repositorio (árbol de carpetas)

El repositorio smartsafe está organizado por áreas de trabajo, cada una a cargo de un integrante del equipo.

3.1 Árbol raíz del repositorio



3.2 Árbol del módulo backend/



```

├── public/
│   ├── images/
│   ├── javascripts/
│   ├── stylesheets/
│   │   └── style.css
│   └── uploads/           → fotografías de eventos (ignorado en git, con .gitkeep)
├── routes/
│   └── index.js           → rutas GET /, POST / y POST /api/eventos
├── services/
│   └── evento.service.js  → logica de negocio compartida (registrarEvento)
├── test/
│   └── evento.test.js     → 9 pruebas unitarias (Jest + Supertest)
├── views/
│   ├── index.ejs          → formulario de reporte
│   ├── correcto.ejs       → confirmacion de registro exitoso
│   └── error.ejs          → pagina de error generica
├── .env                   → credenciales locales (NO se sube a git)
├── .gitignore
├── app.js                 → configuracion de Express (sin listen)
├── package.json
└── package-lock.json

```

4. Git — árbol de ramas y flujo de trabajo

El equipo trabaja con una rama por integrante (prefijo feature/), que se integra mediante Pull Requests. A continuación, el árbol de ramas identificado en el repositorio.

4.1 Árbol de ramas del repositorio

Ramas principales identificadas en GitHub

```

main                                     ← rama estable / produccion
├── develop                             ← integracion del equipo (varios PRs se mergean aqui)
│   ├── feature/user-login
│   ├── feature/report-incident
│   ├── feature/frontend-foundation
│   ├── feature/backend-ci
│   ├── feature/database-foundation
│   ├── feature/initial-data-model
│   ├── feature/docker-compose-integration
│   ├── feature/backend-testing-foundation
│   ├── fix/auth-docker-jwt
│   └── feature/swagger-openapi
├── feature/rony-modulos                ← rama de Rony (este informe)
│   └── Pull Request #38 → base: main

```

Punto abierto para coordinar con el equipo: la mayoría de los Pull Requests observados se integran hacia develop, mientras que el PR #38 (de este módulo) compara contra main. Se recomienda confirmar con el equipo si corresponde cambiar la rama base del PR #38 a develop para mantener un flujo consistente.

4.2 Comandos de Git utilizados en este módulo

Flujo seguido para integrar el módulo backend a la rama feature/rony-modulos:

Clonar y ubicar la rama de trabajo

```

git clone https://github.com/oralb-sys/smartsafe.git
cd smartsafe
git fetch origin

```

```
git checkout feature/rony-modulos
```

Actualizar la rama con los cambios de main

```
git pull origin main
# Si se abre el editor Vim para el mensaje de merge:
#   Esc → escribir :wq → Enter
```

Registrar y subir cambios

```
git status
git add .
git commit -m "mensaje descriptivo del cambio"
git push origin feature/rony-modulos
```

Sacar del control de versiones archivos que no deben subirse (ej. fotos de prueba)

```
git rm --cached public/uploads/*.jpg
# .gitignore ya excluye posteriores archivos en public/uploads/
```

4.3 Comandos para que el equipo clone y configure el proyecto

Requisitos previos: Git y Node.js 18 o superior instalados.

Clonar el repositorio

```
git clone https://github.com/oralb-sys/smartsafe.git
cd smartsafe
git fetch origin
git branch -a
```

Cambiar a una rama específica (ejemplo: la de Rony)

```
git checkout feature/rony-modulos
```

Instalar dependencias y configurar variables de entorno

```
cd backend
npm install
copy .env.example .env      (Windows)
cp .env.example .env        (Mac / Linux)
# Editar .env con los datos propios de conexion a MariaDB
```

Ejecutar las pruebas unitarias

```
npm test
```

Levantar el servidor local

```
npm start
# Abrir http://localhost:3000
```

5. Base de datos

Motor: MariaDB. La conexión se gestiona mediante un pool centralizado y reutilizable, implementado como una clase con una única instancia compartida en toda la aplicación.

5.1 Tablas utilizadas por el módulo backend

Tabla tUsuario

Columna	Tipo (uso)	Descripción
id	INT / PK	Identificador único del usuario
dni	VARCHAR	Documento de identidad — clave de búsqueda
nombres	VARCHAR	Nombres del usuario
apellidos	VARCHAR	Apellidos del usuario

Tabla tEvento

Columna	Tipo (uso)	Descripción
id	INT / PK	Identificador único del evento
id_usuario	INT / FK → tUsuario.id	Usuario que reporta el evento
tipo	VARCHAR	Tipo de evento (Robo, Incendio, Accidente, etc.)
id_personal	INT / FK (nullable)	Personal de seguridad asignado (NULL al crear)
descripcion	TEXT	Descripción del evento reportada por el usuario
foto_path	VARCHAR	Ruta relativa de la fotografía almacenada
coordenadas	POINT (geoespacial)	Ubicación GPS, generada con ST_GeomFromText
estado	VARCHAR	Estado del evento (por defecto: 'Pendiente')
fue_atendido	BOOLEAN	Indicador de atención del evento (por defecto: FALSE)

5.2 Capa de conexión (pool reusable)

La clase MariaDB centraliza la configuración y expone un pool único (patrón similar a Singleton), evitando abrir una conexión nueva en cada petición.

bin/DBconnection.js – configuración del pool

```
static config = {
  host: process.env.MARIA_HOST,
  user: process.env.MARIA_USER,
  password: process.env.MARIA_PASSWORD,
  database: process.env.MARIA_DATABASE,
  connectionLimit: 10,
  acquireTimeout: 10000,
  connectTimeout: 10000,
  bigNumberStrings: true
};

async connect() {
  if (!MariaDB.pool) {
    MariaDB.pool = mariadb.createPool(MariaDB.config);
  }
  return MariaDB.pool;
}

async executeQuery(sql, params = []) { /* SELECT con conexion del pool */ }
async execute(sql, params = [])      { /* INSERT/UPDATE con conexion del pool */ }
async getConnection()                 { /* conexion manual, para transacciones */ }
```

El pool se inicializa una sola vez, al arrancar el servidor (bin/www), antes de aceptar peticiones HTTP:

```
bin/www — inicialización de la base de datos
(async () => {
  try {
    await db.connect();
    console.log('DB lista');
  } catch (err) {
    console.error(err);
    process.exit(1);
  }
})();

server.listen(port, '0.0.0.0', () => { /* ... */ });
```

6. Código fuente del módulo backend

Estado actual (rama feature/rony-modulos), tras la refactorización que separa la lógica de negocio del enrutamiento HTTP.

6.1 Capa de servicio — services/evento.service.js

Contiene toda la lógica compartida entre la ruta web y la API REST: validaciones, búsqueda de usuario y registro del evento. Lanza un error tipado (ValidationError) que cada ruta traduce a su propio formato de respuesta (HTML o JSON).

```
services/evento.service.js (extracto)
class ValidationError extends Error {
  constructor(status, mensaje) {
    super(mensaje);
    this.status = status;
  }
}

async function registrarEvento({ usuario, tipo, descripcion, latitud, longitud, archivo }) {

  if (!usuario) throw new ValidationError(400, 'Debe ingresar el DNI del usuario.');
```

```
  if (!tipo) throw new ValidationError(400, 'Debe seleccionar el tipo de evento.');
```

```
  if (!descripcion) throw new ValidationError(400, 'Debe ingresar una descripcion.');
```

```
  if (!latitud || !longitud)
    throw new ValidationError(400, 'No se pudieron obtener las coordenadas
GPS.');
```

```
  if (!archivo) throw new ValidationError(400, 'Debe adjuntar una fotografia.');
```

```

  const usuarios = await db.executeQuery(
    'SELECT id, nombres, apellidos FROM tUsuario WHERE dni = ? LIMIT 1',
    [usuario]
  );

  if (usuarios.length === 0)
    throw new ValidationError(404, 'El DNI ingresado no corresponde a un usuario
registrado.');
```

```

  const fotoPath = '/uploads/' + archivo.filename;
  const punto = `POINT(${longitud} ${latitud})`;

  const resultado = await db.execute(
    `INSERT INTO tEvento (id_usuario, tipo, id_personal, descripcion, foto_path,
```

```

        coordenadas, estado, fue_atendido)
    VALUES (?, ?, NULL, ?, ?, ST_GeomFromText(?), 'Pendiente', FALSE)`,
    [usuarios[0].id, tipo, descripcion, fotoPath, punto]
);

return {
  idEvento: resultado.insertId,
  fotoPath,
  usuario: { dni: usuario, nombres: usuarios[0].nombres, apellidos: usuarios[0].apellidos }
};
}

```

6.2 Capa de rutas — routes/index.js

Cada ruta delega la lógica en registrarEvento() y solo decide el formato de la respuesta.

```

routes/index.js (extracto – ambas rutas usan el mismo servicio)
// Ruta web: responde HTML
router.post('/', upload.single('foto'), async (req, res, next) => {
  try {
    const resultado = await registrarEvento({ ...req.body, archivo: req.file });
    res.status(201).render('correcto', {
      nombres: resultado.usuario.nombres,
      apellidos: resultado.usuario.apellidos,
      insertId: resultado.idEvento
    });
  } catch (error) {
    if (error instanceof ValidationError)
      return res.status(error.status).send(error.message);
    next(error);
  }
});

// API REST: responde JSON, reutilizable por apps externas
router.post('/api/eventos', upload.single('foto'), async (req, res, next) => {
  try {
    const resultado = await registrarEvento({ ...req.body, archivo: req.file });
    return res.status(201).json({
      success: true,
      mensaje: 'Evento registrado correctamente.',
      id_evento: Number(resultado.idEvento),
      usuario: resultado.usuario,
      foto: resultado.fotoPath
    });
  } catch (error) {
    if (error instanceof ValidationError)
      return res.status(error.status).json({ success: false, mensaje: error.message });
    next(error);
  }
});

```

7. API REST — documentación de endpoints

El endpoint /api/eventos permite que otras aplicaciones (por ejemplo, un script en Python) registren eventos sin pasar por el formulario web.

Método	Ruta	Body (multipart/form-data)	Respuesta éxito	Respuestas de error
--------	------	----------------------------	-----------------	---------------------

GET	/	—	200 — HTML (formulario)	—
POST	/	usuario, tipo, descripcion, latitud, longitud, foto	201 — HTML (vista correcto.ejs)	400 (validación) · 404 (D registrado)
POST	/api/eventos	usuario, tipo, descripcion, latitud, longitud, foto	201 — JSON { success, id_evento, usuario, foto }	400 / 404 — JSON { success:false, mensaje }

7.1 Ejemplo de respuesta exitosa de /api/eventos

201 Created

```
{
  "success": true,
  "mensaje": "Evento registrado correctamente.",
  "id_evento": 25,
  "usuario": {
    "dni": "12345678",
    "nombres": "Rony",
    "apellidos": "Villafuerte"
  },
  "foto": "/uploads/evento-1789252781951.jpg"
}
```

8. Pruebas unitarias

Framework: Jest + Supertest. La base de datos se simula (mock) sobre bin/db.js, por lo que las pruebas no requieren una conexión real a MariaDB ni datos previos cargados.

8.1 Comando para ejecutar las pruebas

```
Dentro de backend/
npm test
```

8.2 Casos cubiertos (9 en total)

#	Endpoint	Caso	Resultado esperado
1	POST /	Registro correcto (DNI válido, todos los campos)	201
2	POST /	Falta el DNI	400
3	POST /	Falta el tipo de evento	400
4	POST /	Faltan coordenadas GPS	400
5	POST /	Falta la fotografía	400
6	POST /	DNI no registrado en tUsuario	404
7	POST /api/eventos	Registro correcto → JSON success:true	201
8	POST /api/eventos	Falta el DNI → JSON success:false	400
9	POST /api/eventos	DNI no registrado → JSON success:false	404

8.3 Resultado de la última ejecución

```

npm test
PASS test/evento.test.js
  POST / (registrar evento - web)
    ✓ debe registrar un evento correctamente
    ✓ debe fallar si falta el DNI (usuario)
    ✓ debe fallar si falta el tipo de evento
    ✓ debe fallar si faltan coordenadas GPS
    ✓ debe fallar si no se adjunta foto
    ✓ debe responder 404 si el DNI no existe en tUsuario
  POST /api/eventos (registrar evento - API JSON)
    ✓ debe registrar un evento y devolver JSON con success:true
    ✓ debe devolver JSON con success:false si falta el DNI
    ✓ debe devolver 404 en JSON si el DNI no existe

Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total

```

Pendiente recomendado: medir cobertura real con "npx jest --coverage" para poder sustentar el criterio de la rúbrica con un porcentaje, no solo con la cantidad de pruebas.

9. Integración continua (GitHub Actions)

Un workflow ejecuta automáticamente npm test dentro de backend/ en cada push y en cada Pull Request hacia main, sin que nadie deba instalar nada manualmente.

```

.github/workflows/tests.yml
name: Pruebas unitarias - SmartSafe

on:
  push:
    branches: [ "*" ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: npm install
      - run: |
          echo "MARIA_HOST=localhost" >> .env
          echo "MARIA_USER=test" >> .env
          echo "MARIA_PASSWORD=test" >> .env
          echo "MARIA_DATABASE=test" >> .env
      - run: npm test

```

Punto abierto para coordinar con el equipo: existe un segundo workflow ("Backend CI") que corre sobre develop y ramas feature/, de autoría aún por confirmar. Se recomienda revisar en conjunto si conviene fusionarlos en uno solo o mantenerlos separados con nombres claros, para no duplicar ejecuciones.*

10. Despliegue

La aplicación está desplegada y accesible en: daii.unsaac.edu.pe/smartsafe

10.1 Estado actual del despliegue

- El proceso Node.js se mantiene activo en el servidor mediante PM2 (gestor de procesos: reinicia la aplicación si se cae).
- La actualización del código en el servidor es manual: se requiere ingresar y ejecutar git pull seguido de un reinicio con PM2 (pm2 restart) cada vez que hay cambios nuevos.
- No existe, por ahora, un paso automatizado que despliegue el código apenas se aprueba un Pull Request (no hay entrega continua / CD).

Esta distinción es relevante para la rúbrica: PM2 automatiza que la aplicación se mantenga corriendo, pero no automatiza que el código se actualice — son dos cosas distintas. Ver sección 11.4.

11. Rúbrica de evaluación — estado actual

Evaluación honesta del módulo backend (Rony) frente a la rúbrica del curso. Los criterios de Gestión y Optimización dependen del equipo completo o no tienen evidencia disponible todavía.

Categoría	Criterio	Nivel alcanzado	% estimado
Código	Nombres Reveladores	Bueno / cerca de Excelente	~85%
Código	Patrones de Diseño	Aceptable	~60%
Proceso	Pruebas Automatizadas	Aceptable	~55-60%
Proceso	Control de Versiones y Entrega	Aceptable	~55-60%
Gestión	Planificación y Equipo	Sin datos suficientes	—
Optimización	Criterio de Optimización	No aplica aún	—
Integración API	Verbos HTTP y Códigos de Estado	Bueno	~75-80%

11.1 Nombres Reveladores (~85%)

Nombres de variables y funciones claros en todo el módulo: usuario, idUsuario, fotoPath, registrarEvento(). Se entiende la intención del código sin necesitar comentarios adicionales.

11.2 Patrones de Diseño (~60%)

Mejóro con la separación en capas (routes → services → bin/db). La clase MariaDB actúa como un pool centralizado tipo Singleton. Pendiente: separar una capa de "repositorio" explícita, y usar consistentemente las vistas EJS en todas las respuestas.

11.3 Pruebas Automatizadas (~55-60%)

Existe una suite real de 9 pruebas que corre en cada push vía CI. Falta: medir la cobertura real (%) y ampliar la suite a otros módulos del sistema, no solo al de eventos.

11.4 Control de Versiones y Entrega (~55-60%)

El uso de Git es sólido: ramas por integrante, Pull Requests, CI corriendo pruebas automáticamente. El nivel baja porque el despliegue a producción sigue siendo manual (ver sección 10) — la rúbrica exige entrega continua automatizada para el nivel más alto.

11.5 Planificación y Equipo (sin datos suficientes)

El repositorio contiene docs/product-backlog.md, lo que sugiere que existe algún nivel de planificación, pero su contenido no ha sido revisado en conjunto con todo el equipo para esta evaluación.

11.6 Optimización (no aplica aún)

No hay evidencia de optimización prematura ni basada en un profiler — lo cual es correcto para el tamaño y la etapa actual del proyecto. No hay nada que corregir aquí todavía.

11.7 Integración API (~75-80%)

Los códigos de estado son precisos: 201 en creación, 400 en validaciones, 404 cuando el DNI no existe. Solo se usan GET y POST porque el alcance definido por el docente no requiere edición ni eliminación de recursos — no es una omisión, es una decisión de alcance justificada.

12. Pendientes y recomendaciones para el equipo

Punto	Acción recomendada
Rama base del PR #38	Confirmar si debe apuntar a develop en lugar de main, para seguir el mismo flujo que el resto del equipo.
Workflow de Actions duplicado	Definir en conjunto si "Backend CI" y "Pruebas unitarias - SmartSafe" se fusionan o se mantienen separados con nombres claros.
Despliegue automático (CD)	Evaluar agregar un paso en GitHub Actions que haga el despliegue automático tras un merge a main (git pull + pm2 restart vía SSH), si el tiempo del curso lo permite.
Cobertura de pruebas	Ejecutar npx jest --coverage y documentar el porcentaje real obtenido.
Planificación del equipo	Revisar en conjunto docs/product-backlog.md y confirmar que refleje historias de usuario reales.
Patrones de diseño en el resto del sistema	Extender la separación en capas (rutas / servicio / datos) a los demás módulos del proyecto, no solo al de eventos.

13. Conclusiones

- El módulo backend de registro de eventos está completo, probado (9/9 pruebas) y desplegado en producción.
- El flujo de Git (ramas, Pull Request, CI) sigue buenas prácticas profesionales, con un punto de mejora claro en la automatización del despliegue.
- La refactorización a una capa de servicio compartida (services/evento.service.js) resolvió la duplicación de código entre la ruta web y la API REST, mejorando el criterio de patrones de diseño.
- Los puntos pendientes identificados en este informe (sección 12) son la base de la conversación que el equipo debe tener antes de la entrega final.